

Received 8 October 2025, accepted 8 December 2025, date of publication 15 December 2025,
date of current version 31 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3644152

RESEARCH ARTICLE

A Time-Triggered Edge-Fog-Cloud Architecture With Hierarchical Genetic Optimization for Safety-Critical Applications

JOSEPAUL PAULACHAN¹, OMAR HEKAL¹, DANIEL ONWUCHEKWA¹,
AND ROMAN OBERMAISSER¹

Chair for Embedded Systems, University of Siegen, 57076 Siegen, Germany

Corresponding author: Josepaul Paulachan (josepaul.paulachan@uni-siegen.de)

This work was supported by the EdgeAI-Trust project, which is funded within the Chips Joint Undertaking (Chips JU), a public-private partnership in collaboration with the European Union's Horizon Europe research and innovation program and National Authorities under grant agreement number 101139892.

ABSTRACT The growth of safety-critical Internet of Things (IoT) applications, such as autonomous driving and industrial automation, demands predictable, real-time performance from distributed Edge-Fog-Cloud systems. Existing scheduling algorithms for this continuum are fundamentally unsuitable for safety-critical domains. These algorithms primarily use best-effort communication networks. This results in non-deterministic latencies and an inability to provide temporal guarantees. This paper introduces a new solution that bridges the gap between task scheduling and network determinism: a Time-Triggered Edge-Fog-Cloud (TTEFC) architecture that is based on Time-Sensitive Networking (TSN) and Deterministic Networking (DetNet) standards, ensuring bounded end-to-end communication delays; and a Hierarchical Nested Genetic Algorithm (HNGA) that effectively maps the scheduling problem onto this three-tier architecture. The HNGA consists of two genetic algorithms that use a feedback mechanism. At the top level, it is called the global genetic algorithm (GGA), which utilises graph partitioning techniques to partition the task graph into latency-aware subgraphs and assign them across the continuum. Meanwhile, nested local genetic algorithms (LGA) receive these subgraphs and perform fine-grained, time-triggered scheduling within each tier. The LGA provides the update back to the GGA, which uses the information to optimise the subgraphs such that deadlines are met and also generates the global schedule for the safety-critical applications. Experimental validation shows that this integrated approach significantly outperforms state-of-the-art scheduling methods. The proposed method was compared with the Min-Min, Particle Swarm Optimisation (PSO), and Heterogeneous Earliest Finish Time (HEFT) algorithms. The results show that the proposed method significantly reduces the makespan and consistently meets deadlines when compared to the above algorithms. Thus, the main contribution of this work is a robust and verifiable framework that provides a comprehensive solution for deploying safety-critical, real-time applications across the Edge-Fog-Cloud continuum.

INDEX TERMS Algorithm, cloud computing, deterministic communication, distributed systems, edge-fog-cloud computing, edge computing, fog computing, genetic algorithm, graph partitioning, multi-tier architecture, real-time, reliability, safety-critical, task scheduling, time-triggered, TSN communication.

I. INTRODUCTION

A significant shift is transforming the distributed computing as applications like autonomous driving, industrial

automation, and smart manufacturing are increasingly deployed across the Edge-Fog-Cloud (EFC) architecture. Unlike traditional software, the above applications are now designed as distributed task graphs, where individual components get executed across multiple layers of the EFC continuum. For example, time-critical sensor fusion and

The associate editor coordinating the review of this manuscript and approving it for publication was Zhenliang Zhang.

immediate control are handled at the edge while the data analysis or collection occurs in the fog, and computationally intensive AI model training or large-scale optimisation runs in the cloud [1]. This decomposition is motivated by the dual need for local responsiveness and scalable, high-throughput data processing.

However, all the applications discussed above have a safety-critical nature. Thus, their correct execution depends not only on computational correctness but also on strict and predictable timing [2]. A delayed response in any of these applications can result in accidents. For instance, consider the autonomous driving application in which a delay in a response could result in fatal accidents. Similarly, in the case of industrial automation, a missed control signal could stop production lines or damage equipment. The EFC architecture on which these applications are deployed offers extremely powerful capabilities in terms of distributed processing and scalability. However, they usually rely on best-effort communication networks, which cannot provide deterministic guarantees, leaving a significant gap between computational capability and real-world reliability.

This mismatch between the architecture and the application running occurs because most of the conventional scheduling methods in the EFC continuum simplify the communication as a fixed latency cost. They do not consider the non-determinism, jitter, and congestion in a real network. However, in case of the safety-critical applications, these oversights are unacceptable as a single delayed message or missed deadline can lead to catastrophic system failures. The flexibility and scalability which make EFC architecture attractive simultaneously make it not suitable for time-sensitive operations.

The different deterministic networking standards like Time-Sensitive Networking (TSN) and Deterministic Networking (DetNet) [3], [4] promise to resolve these issues at the network layer. These standards can provide bounded latency, minimal jitter, and synchronised timing. Thus, they transform an unpredictable network into a reliable, deterministic communication network. However, their application has been primarily focused on edge layer rather than on multi-tier computing continuum. This has created a critical disconnect between the application layer's need for end-to-end predictability and the network layer's capability to provide it. There is a clear lack of an integrated framework that co-designs the system architecture for determinism and the scheduling algorithm for hierarchy-awareness.

To bridge this gap, this paper proposes a novel, time-triggered framework for safety-critical systems on the Edge-Fog-Cloud continuum. This framework is built on two core, co-designed contributions. First, a Time-Triggered Edge-Fog-Cloud (TTEFC) architecture, which uses a TSN/DetNet fabric to provide bounded end-to-end communication delays. This helps to transform the non-deterministic EFC continuum into a predictable, time-triggered system suitable for safety-critical applications. Second, a Hierarchical Nested Genetic Algorithm (HNGA), which is specifically designed to exploit

the TTEFC architecture so that it places and coordinates tasks correctly while fully exploiting the deterministic capabilities of the network. The HNGA is made of a two-level structure to mirror the EFC continuum: a Global Genetic Algorithm (GGA) partitions the application task graph and maps subgraphs to the appropriate tier (Edge, Fog, or Cloud) based on latency and computational needs, while Local Genetic Algorithms (LGAs) perform fine-grained, time-triggered scheduling of individual tasks within each tier to meet the deadlines. Both algorithms communicate via a feedback mechanism to optimise the partitions and generate a global schedule that satisfies the application's deadline. The main novelty lies in the synthesis of a time-triggered cloud-fog-edge continuum with an architecture-level fault tolerance, supported by TSN/DetNet, with a scheduler that is structurally aware of this hierarchy.

Our approach addresses the key challenges faced in scheduling within a distributed systems. The hierarchical nature of the proposed HNGA manages the complexity and resource heterogeneity in the three-tier continuum thus ensuring both scalability and efficiency. By building upon a deterministic network fabric, the framework provides the verifiable temporal guarantees that are non-negotiable for safety-critical applications. The main contributions of this work are as follows:

- A novel TTEFC architecture that incorporates a TSN/DetNet fabric, ensuring bounded end-to-end communication delays across all layers.
- A co-designed HNGA featuring a two-level optimisation structure that maps directly to the three-tier TTEFC architecture, optimising both task allocation and scheduling concurrently.
- A holistic, time-triggered architecture that bridges the gap between application-level scheduling and network-level determinism, providing a comprehensive solution for real-time, safety-critical systems.
- An extensive experimental evaluation demonstrates the proposed architecture's superior performance in reducing makespan compared to established scheduling algorithms.

The remainder of this paper is structured as follows: Section II reviews related works. Section III details the proposed system model, including the TTEFC architecture and the formal optimisation problem. Section IV presents the experimental setup, followed by a discussion of the results in Section V. Finally, Section VI concludes the paper and outlines future research directions.

II. RELATED WORKS

A. CLOUD-FOG-EDGE ARCHITECTURE

The paper proposed by Kochovski et al. [5] presents a decision-making method and architecture for placing database containers across the Edge-Fog-Cloud continuum. The proposed architecture automates the container placement and orchestration process, addressing the complexity

of multi-criteria decision-making in distributed systems. It provides formal verification and Quality of Service (QoS) probability assurances, aiding software engineers in making optimal deployment decisions. The architecture's effectiveness is demonstrated through experiments involving different workloads and computing infrastructures. It highlights its suitability for intelligent applications in IoT and AI domains that require efficient and reliable data management across varied infrastructure environments.

The paper from Tong et al. [6] proposes a hierarchical edge cloud architecture designed to improve the performance of mobile computing by efficiently managing peak loads and enhancing resource utilisation. The authors introduce a multi-tiered structure for edge cloud servers, enabling more effective aggregation and handling of computational loads across different levels. They also develop workload placement algorithms that dynamically allocate mobile workloads to various server tiers, minimising execution delays. Their analytical models and evaluations, based on both small-scale experiments and large-scale simulations, demonstrate that the hierarchical architecture significantly outperforms traditional flat designs, particularly in reducing average program execution delays by over 25%.

The paper from Mohan and Kangasharju [7] presents an architecture called the Edge-Fog cloud, which integrates Edge and Fog computing models to handle the distributed nature of IoT data better. This architecture consists of three layers: the Edge layer, comprised of loosely coupled, human-operated devices close to IoT sensors; the Fog layer, which includes high-performance networking devices managed by cloud vendors; and a central Data Store for reliable data archiving. The authors also introduce the LPCF (Least Processing Cost First) algorithm, designed for efficient task assignment, minimising processing time and network costs.

The paper [8] discusses the integration of cloud computing techniques with the Internet of Things (IoT) to provide valuable and reliable services to users. It presents an architecture for an IoT cloud, highlighting the combination of HTTP and MQTT servers as well as the implementation of a message broker. Experimental results demonstrate the performance of the IoT cloud under various conditions, emphasising factors such as the number of clients and CPU cores on transmission latency, throughput, and CPU utilisation.

The paper [9] explores the critical aspects of communication protocols in the application layer of the Internet of Things (IoT) architectures, focusing on their integration with fog and cloud computing systems. It identifies the fragmented nature of currently used protocols across various IoT domains due to the diverse requirements, such as functionalities of sensors, actuators, computing power, security, and connectivity. Key protocols analysed include HTTP, CoAP, MQTT, AMQP, XMPP, and DDS, each with unique strengths and challenges. The paper highlights the lack of a standardised communication protocol as a significant challenge, emphasising the need for protocols that

balance performance and security, particularly in resource-constrained settings. The study advocates for the careful selection and hybrid use of communication protocols to achieve seamless interoperability and integration within IoT, fog, and cloud systems, calling for further research to optimise these protocols for IoT architectures effectively.

B. REAL-TIME, TSN, FAULT TOLERANT CLOUD-FOG-EDGE

In the realm of real-time processing architectures for cloud-based systems, Khriji et al. [10] introduced the Real-time processing Event-Driven cloud-based Architecture (REDA), drawing inspiration from the Amazon Web Service IoT core architecture. This framework is designed to work on an event-driven model using the MQTT protocol for message transmission. The authors have used Apache Kafka as a message broker for real-time stream processing. Their evaluation focuses on assessing the architecture's performance regarding availability, reliability, throughput, and latency, demonstrating its capability to handle a significant volume of messages with minimal delay.

The paper from Malik and Huet [11] proposes a model that enables a system to adapt to faults by making decisions based on the reliability of processing nodes, namely virtual machines (VMs) in cloud computing infrastructure. This adaptive reliability mechanism adjusts after each computing cycle, with VM reliability increasing if correct results are produced within the allotted time and decreasing if failures occur. To ensure fault tolerance, software variants run on multiple VMs, which increases cloud resource costs but mitigates catastrophic losses. The scheme selects VMs for computation based on their reliability, with under-performing VMs subject to removal from real-time applications.

Koubaa and Qureshi [12], in their work, introduce DroneTrack, a real-time object tracking system utilising drones and cloud-based infrastructure. Central to their approach is the Dronemap planner (DP), a cloud-based system that facilitates drone control, communication, and management over the Internet. The primary contributions of the paper include the development and deployment of DroneTrack, as well as an extensive experimental evaluation of its real-time performance. The tracking mechanism relies on GPS location exchange rather than computer vision techniques. Specifically, the paper details the design and architecture of DroneTrack, emphasising its cloud-based nature and the utilisation of UAVs for real-time object tracking. The connectivity between drones and the cloud is established via 3G/4G Internet, utilising the MAVLink protocol for communication.

The paper from Kochar and Sarkar [13] introduces a novel two-level framework for symbiotic sharing and consolidating edge computing resources to create a real-time fog computing environment with high resource utilisation. Furthermore, the framework incorporates profit awareness by ensuring a minimum guaranteed revenue to the fog service provider. The authors conduct simulation-based experiments on diverse datasets, yielding promising results. The authors present an

efficient EDF-based framework for fog computing, including two scheduling algorithms, DEF (Distributed ED for Fog) and PDEF (Profit-aware Distributed EDF for Fog), to maximise resource utilisation.

In this paper [14], the authors introduce a novel utility accrual scheduling algorithm for real-time cloud computing services, aiming to maximise total utility by scheduling tasks non-preemptively. Unlike traditional approaches using a single time utility function (TUF), the proposed algorithm employs two distinct TUFs—a profit TUF and a penalty TUF—for each task, addressing the need to reward early completions while penalising task abortions or deadline misses.

The paper [15] introduces the concept of a time-triggered virtual machine (TTVM) as a critical component in realising a real-time cloud, referred to as a fog, for providing reliable electronic services with temporal guarantees on board vehicles. The TTVM serves as a precisely specified virtual interface between real-time software components and their underlying hardware infrastructure, enabling fault tolerance, evolution, and validation in a time-triggered distributed architecture. The proposed architecture and design methodology are illustrated using a driver assistance system example on board a car.

In this paper [16], the authors tackle the challenge of run-time reconfiguration in TSN-based Fog Computing platforms comprising heterogeneous Fog Nodes interconnected using TSN. The main focus is on handling hard real-time control tasks with strict timing requirements, expressed as hard deadlines. The proposed heuristic algorithm is designed to determine Gate-Control Lists (GCLs) at run-time, ensuring that deadlines are met while minimising queue usage to accommodate non-critical traffic. Integrated into a Configuration Agent, the scheduling algorithm responds to network changes and reconfigures GCLs using the NET-CONF network configuration protocol. This work presents a new approach to re-configuring TSN-based Fog Computing platforms.

C. SCHEDULING IN CLOUD-FOG-EDGE ARCHITECTURE

The paper [17] surveys the various task scheduling algorithms in cloud-fog architecture. The authors say that among the algorithms they investigated, 58% of the scheduling algorithms were static, and the remaining were dynamic. They also say that 53% of them focused on scheduling in fog while the remaining 47% focused on scheduling in a cloud-fog environment. The authors of the paper [18] introduce a scheduling algorithm called Reinforcement Learning Fog Scheduling (RLFS) for task scheduling in edge-fog-cloud architecture. The authors compare the proposed algorithm against GA, PSO-SA, Load Balanced Service Scheduling Approach (LBSSA) and Dynamic Resource Allocation Method (DRAM) and analyse execution time, load balance, number of devices used and resource efficiency. The paper [19] propose a new scheduling algorithm called New Genetic Ant Colony Optimisation (NGACO) for resource

scheduling in fog computing. The authors modify the GA by incorporating a randomised initialisation method and the ACO using a niche technology. The authors use this algorithm to reduce the makespan and total cost and improve load balancing.

The authors of the paper [20] introduce an evolutionary task scheduling algorithm (ETS) for task scheduling in a cloud-fog environment focusing on execution time and operating costs of Bag-of-Tasks (BoT) applications. In paper [21], the authors propose a Priority-aware Genetic Algorithm (PGA) for scheduling tasks on to the heterogeneous resources in a cloud-fog platform. The algorithm uses a hybrid approach called classification and GA to schedule the task onto suitable computational nodes in the platform. The authors of the paper [22] introduce an enhanced discrete non-dominated sorting genetic Algorithm 2 (DNSGA-II) for solving the problem of task scheduling and resource allocation in fog-cloud environments. The paper says that the algorithm achieves lower makespan and costs.

The paper “Towards task scheduling in a cloud-fog computing system” [23] introduces a heuristic-based algorithm for task scheduling in a cloud-fog computing system. The authors propose a two-level algorithm that could balance the makespan and the cost involved in the cloud resources. The authors compare the two-phase algorithm introduced with Greedy for Cost classical heterogeneous earliest-finish-time (HEFT) and Dynamic Level Scheduling (DLS) algorithm. The authors of paper [24] introduce a priority-based task scheduling for fog computing. The introduced algorithm enhances the Efficient Resource Allocation (ERA) algorithm with priority scheduling. The authors also used the CloudAnalyst simulator to evaluate their proposed method.

The paper “Task scheduling in cloud-fog computing systems” [25] introduces two scheduling algorithms for task scheduling in cloud-fog computing systems. The algorithms are based on integer linear programming. The two algorithms try to find a schedule with minimum makespan by taking in the parameters such as CPU, RAM and storage space. The authors compare their proposed algorithms against Random and round-robin algorithms. The authors of the paper “Real-Time Task Scheduling Algorithm for IoT-Based Applications in the Cloud-Fog Environment” [26] introduce a semi-dynamic real-time task scheduling algorithm for IoT applications in cloud-Fog architecture. They modelled the task scheduling as a permutation-based optimisation problem and used a modified genetic algorithm to obtain this permutation at each scheduling round. In paper [27], the authors introduce a graph partitioning-based task scheduling approach in the multi-cloud environment to reduce the overall execution time. The authors use a nested GA to schedule tasks in the multi-cloud environment.

Based on the literature review conducted, the following research gaps were identified. Although many studies have examined task scheduling in Edge-Fog-Cloud architectures, they largely overlook the critical need for deterministic

communication, which is important for safety-critical and real-time systems. Many existing scheduling algorithms reviewed typically use basic latency assumptions to model communication networks, which makes them unsuitable in fields like autonomous driving and industrial automation. As for these fields, predictable timing is not merely a performance metric, but an essential safety requirement. In response to these gaps, this paper addresses these deficiencies by introducing a holistic framework, to our knowledge, the first to combine a hierarchically aware scheduling algorithm with a determinism-focused system architecture. Our main contributions are:

- A Time-Triggered Edge-Fog-Cloud (TTEFC) architecture that natively incorporates a TSN/DetNet fabric, ensuring bounded end-to-end communication delays across all layers
- A Hierarchical Nested Genetic Algorithm (HNGA). The HNGA features a two-level optimisation structure specifically designed to reflect the three-tier TTEFC architecture. At the top level, the Global Genetic Algorithm (GGA) divides the task graph into smaller, latency-aware subgraphs and allocates them across the different architectural layers. At the nested level, the Local Genetic Algorithm (LGA) takes these assignments, performs detailed scheduling within each layer, and offers feedback to refine the global partitioning. Therefore, the proposed hierarchical nested approach ensures that both subgraph allocation and task scheduling are optimised together to meet the timing constraints. Thus, the HNGA method provides a scalable and robust solution to the complex multi-tier scheduling problem.

Our work proposes a scheduling algorithm and system architecture that provide a comprehensive solution, bridging the gap between high-performance distributed computing and the stringent real-time requirements of safety-critical applications.

D. COMPARISON OF PROPOSED METHOD AND OF RELATED WORKS

To provide a clear and analytical overview of the existing literature, we present a comparative analysis of recent scheduling frameworks within the edge-fog-cloud continuum in Table 1.

III. SYSTEM MODEL

This work addresses the challenge of scheduling tasks within a multi-tier heterogeneous Edge-Fog-Cloud (TTEFC) architecture to support real-time applications with strict deadline constraints. The objective is to efficiently distribute tasks across the different layers while minimising execution time (makespan) and efficiently utilising the available resources. To achieve this, we propose a Hierarchical Nested Genetic Algorithm (HNGA) that combines Graph Partitioning (GP) with Genetic Algorithms (GAs), allowing scheduling across the three-tier architecture.

The proposed system model is represented in the figure 1 and the section below explains the proposed TTEFC architecture and HNGA algorithm in detail.

A. CLOUD LAYER

In the proposed TTEFC (Time-Triggered Edge-Fog-Cloud) architecture, the Cloud Layer is treated as a opaque abstraction that provides high-performance computation and real-time guarantees. While the internal workings of the cloud are abstracted, we assume that the cloud can reliably execute safety-critical and real-time tasks while maintaining performance, scalability, and fault tolerance. The Cloud Layer is responsible for cloud computing functionalities. It forms the backbone infrastructure that supports upper-layer services and applications. Although we conceptualise the cloud as a opaque abstraction in our architectural model to simulate and evaluate real-world performance during experiments, we have modelled the cloud environment based on widely-used commercial platforms— Amazon AWS and Google Cloud Platform (GCP). The above platforms offer realistic constraints and configurations that help validate our assumptions about resource availability, execution time, and communication delays in cloud-based deployments of real-time and safety-critical workloads.

B. FOG LAYER

The fog layer in our architecture acts as the central system. Positioned between the Cloud Layer and the Edge layer, the Fog Layer plays a significant role in the TTEFC architecture.

As mentioned above, the fog layer is the orchestrator for all three layers. It is responsible for scheduling, monitoring, adapting, and diagnosing the execution of the task in the three layers. The various components in the fog layer of our architecture include a fog orchestrator, heterogeneous computational nodes (End Systems), TSN switches and storage devices. Each fog in the architecture has the dedicated components mentioned above, along with multiple edge layers connected. Each fog acts as an independent unit and does not share information, meaning the communication happens southbound and northbound. The task offloading happens between the fog and its edge layers and the different clouds available.

The Fog orchestrator performs various activities both at inter and intra level. The activities include scheduling the task graphs to various resources available in the fog layer and also onto edge layers. It is also responsible for offloading the activities to the cloud based on the timing requirement. It also runs diagnostics and re-configurations of various devices based on the needs. It is also responsible for ensuring fault tolerance and reliability across the layers, primarily on the fog and edge levels. It also runs algorithms such as graph partitioning, scheduling, and negotiation algorithms to achieve the timing and meet the deadline for task execution. Figure 2 shows the different components inside the fog orchestrator.

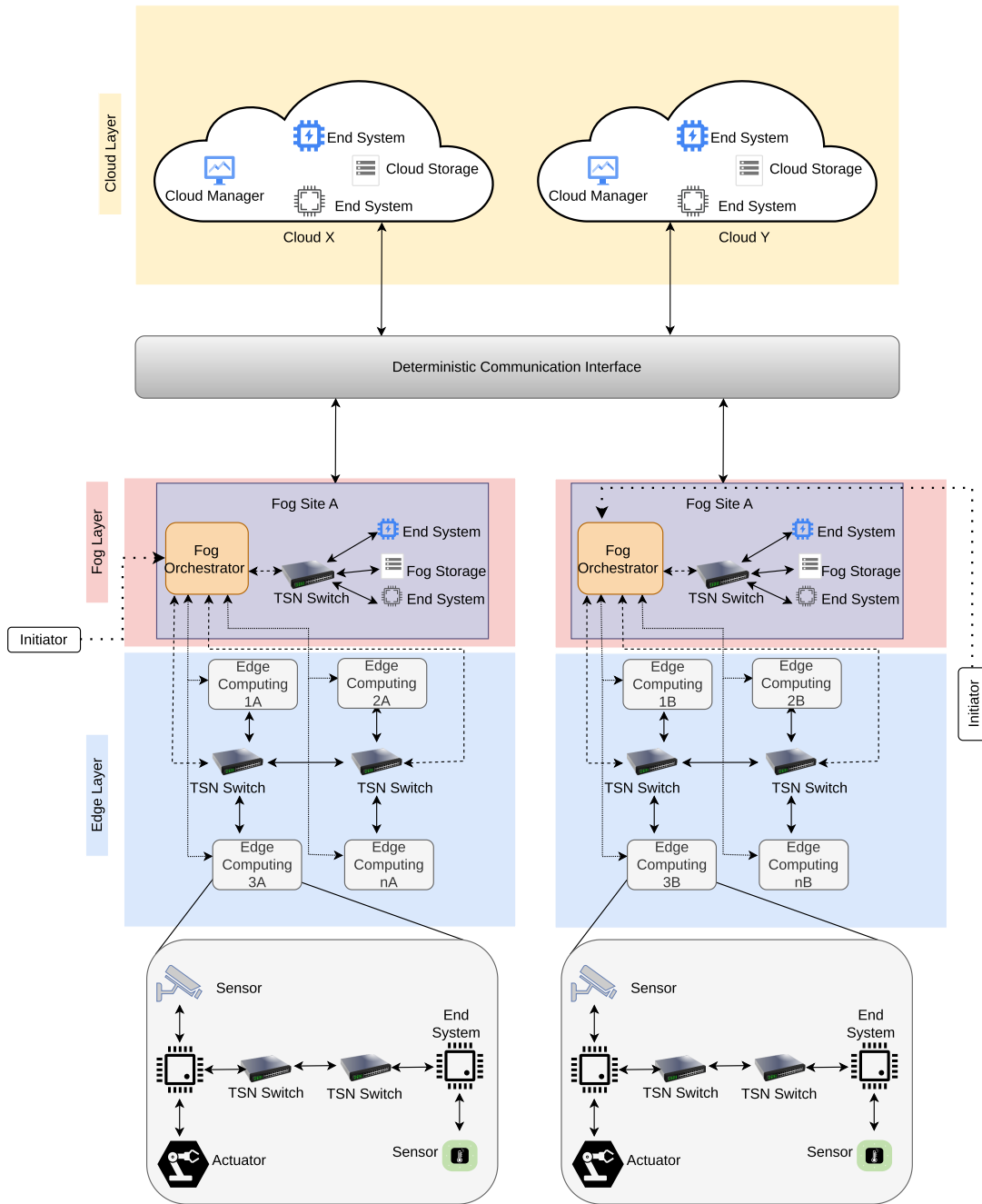


FIGURE 1. Proposed time-triggered edge-fog-cloud architecture.

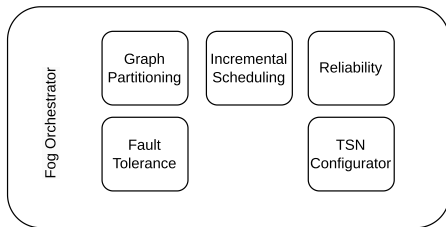
The Fog Orchestrator in the TTEFC framework uses a communicating finite state machine architecture to manage and schedule applications modelled as Directed Acyclic Graphs (DAGs). This architecture comprises several interacting state machines that run in parallel and communicate asynchronously. It allows the system to manage application arrivals, partitioning, and scheduling efficiently. This design helps the proposed system to adapt to changing workloads while ensuring scalability and responsiveness. Figure 3 shows

the internal structure of the state machine used in the Fog Orchestrator.

To understand the operation of the state machine, consider how applications arrive dynamically from multiple Initiator modules (Nodes). When an application (e.g., A_0) arrives at time T_0 , the system activates the *Task Arrival* state, then transitions to the *Admission_Check* state. Here, the system verifies whether the application meets basic criteria such as resource availability, system load, and whether

TABLE 1. Comparison of proposed method and of related works.

Feature	This Work (TTEFC+HNGA)	Pham & Huh [23]	Hoseiny et al. [21]	Abrahamama et al. [26]	Ramezani et al. [18]
Architecture	Edge-Fog-Cloud	Cloud-Fog	Fog-Cloud	Cloud-Fog	Edge-Fog-Cloud
Algorithm Type	Meta-heuristic (Hierarchical GA)	Heuristic	Meta-heuristic (GA)	Meta-heuristic (GA)	Reinforcement Learning
Communication Model	Deterministic (TSN/DetNet)	Not Specified	Not Specified	Not Specified	Not Specified
Real-Time Guarantees	Yes	No	No	Partially	No

**FIGURE 2.** Fog orchestrator.

its deadline can be met. If the application passes the check, it proceeds to the *Submit_for_Scheduling* state, which triggers a *schedule_request!* event. This activates the *Partitioning and Offer Validation* state machine, which breaks the DAG into smaller subgraphs. Once partitioned, the state machine issues a *schedule_subgraph_request!*, triggering the *Scheduling* state machine. The *Scheduling* state machine then maps these subgraphs to available edge and fog resources to generate *Local Offers (LO)*. In parallel, *Cloud Offers (CO)* are generated by cloud platforms treated as external opaque abstraction systems that provide timing guarantees. All *LO* and *CO* are sent back to the *Offers Validation* state in the *Partitioning and Offer Validation* state machine. During this phase, the optimisation loop continues (refer to figures 3, 4 and 5), checking whether the generated schedules meet the application's constraints. This process continues until a termination condition is met, such as convergence or a maximum number of iterations. If a valid scheduling plan is found, the *Confirm_Schedules!* event is triggered, transitioning the system to the *Dispatch* state, where the selected execution plan is deployed and then dispatched. Suppose the offers are invalid or the application was rejected at admission. In that case, the system transitions to the *Notify_Application* state, informing the user of the failure. The figure 4 shows the processing path involved.

As shown in the timeline diagram (Figure 5), multiple applications such as $A_0, A_1, A_2, \dots, A_n$ can arrive at different time instances $(T_0, T_{0+1}, T_{0+2}, \dots, T_n)$.

Each application is managed by its instance of the state machines, allowing them to run concurrently without interference. This modular and scalable design ensures that the TTEFC system can effectively handle high-throughput, real-time workloads, even under heavy system load or frequent application arrivals.

1) GRAPH PARTITIONING

One of the primary roles of the fog orchestrator is graph partitioning, which helps distribute tasks efficiently across the edge, fog, and cloud layers in the proposed architecture. The orchestrator first analyses the application DAG to understand the data dependencies between tasks and any specific execution requirements. For example, consider an application as shown in Figure 6, where task T_b depends on the output of task T_a . In this case, T_b can only begin once T_a is finished, which shows the data dependency between the two tasks. If T_a is a data collection task, it has an execution requirement to run closer to the data source so the orchestrator assigns it to an edge device to reduce latency and enable real-time processing. By identifying the data dependencies and execution constraints, the graph partitioning module splits the DAG into smaller parts that can be processed in parallel. These parts are then further optimised using algorithms like Genetic Algorithms (GA), which help reduce the total execution time (makespan), meet real-time deadlines, and make the best use of the different computing resources available in the edge-fog-cloud setup.

2) INCREMENTAL SCHEDULING

Another critical component of the fog orchestrator is the incremental scheduler. It schedules the new applications arriving without significantly changing the previous schedule. It leverages incremental scheduling approaches [28], [29] introduced in previous works to allocate resources while maintaining the stability of the base schedule. This scheduling mechanism ensures that newly admitted applications are assigned to available computational resources in a way that minimises disruptions to ongoing tasks. Additionally, the scheduler prioritises deadline adherence, ensuring that each application completes execution within specified time constraints. This method reduces the computational overhead by incrementally adjusting the schedule rather than performing a complete rescheduling. Thus making it well-suited for scheduling real-time and time-sensitive applications for the TTEFC architecture.

3) TIME SENSITIVE NETWORKING CONFIGURATOR

In the fog orchestrator, the TSN configurator is responsible for setting up the TSN devices in the architecture. It is designed to manage and set up key network parameters

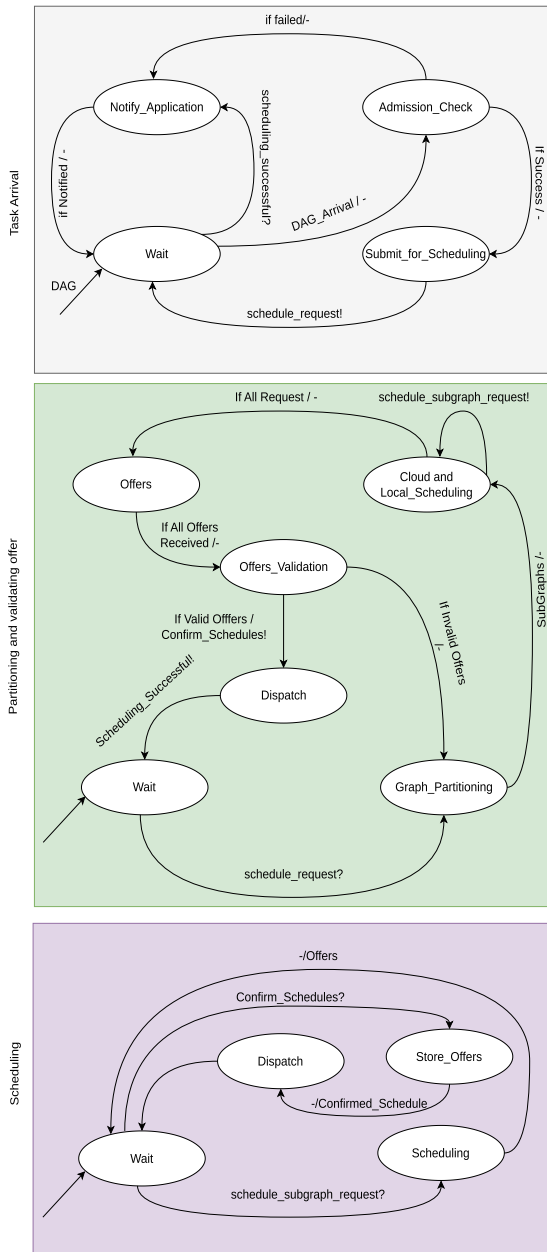


FIGURE 3. Communicative finite state machine - fog orchestrator.

such as configuring the Gate Control List (GCL) for precise scheduling and ensuring time synchronisation across network devices using IEEE 802.1AS standards. Thus, the TSN configurator ensures that networks meet stringent performance and reliability requirements.

4) RELIABILITY

Reliability of a system can be defined as the dependability and consistency of a system or a task to perform a function under specific conditions over time [30]. It is important

in many sectors especially in real-time and safety-critical systems. Hence, we integrate this feature into our architecture as the proposed architecture is intended to be used for running real-time and safety-critical applications.

The reliability of a system $R(t)$ is calculated using the probability of success as:

$$R(t) = e^{-\lambda t} \quad (1)$$

where λ is failure rate. However, in a typical system like the proposed TTEFC, we have parallel, series and mixed configurations. Thus we calculate the reliability of the system R_{system} for each module i using the equation 2, [31], [32]. We calculate the reliability for devices at each layer and also the global reliability. The detailed reliability modelling and quantitative evaluation of the TTEFC framework are part of future work; the architecture itself provides the necessary deterministic properties to enable such analysis.

$$R_{\text{system}} = R_i \cdot \text{Prob}\{\text{System works} | i \text{ is fault-free}\} + (1 - R_i) \cdot \text{Prob}\{\text{System works} | i \text{ is faulty}\} \quad (2)$$

5) FAULT TOLERANCE

Another key component of the fog orchestrator is the fault-tolerant module. It ensures that our TTEFC architecture is reliable and resilient against failures. The fault-tolerant module consists of two primary sub-modules: the Diagnosis Unit and the Adaptation Unit.

The Diagnosis Unit monitors the system for faults, errors, or performance degradation. When a fault is detected, the Adaptation Unit is notified to take corrective measures. The Adaptation Unit dynamically adjust the current schedule by using the meta-scheduling approach, which relies on pre-computed backup schedules to ensure that the fault is contained in the fault containment region. The full implementation and experimental evaluation of these fault tolerance mechanisms are planned for future work.

C. EDGE LAYER

The Edge Layer in the TTEFC architecture is the first point of interaction between the physical world. It is the closest layer to users/ data sources such as sensors and IoT devices. Our proposed architecture has multiple Edge Layers, offering a robust distributed approach. This configuration provides several advantages, especially for safety-critical and real-time task execution. By distributing the workload across multiple Edge Layers, the proposed architecture can ensure continuous operation even if one Edge Layer encounters issues, thereby increasing overall reliability. The architecture can quickly scale up by adding more Edge Layers under a Fog Layer, for accommodating growing data volumes and computational tasks without overloading any single component. Tasks can be efficiently distributed across multiple Edge Layers, optimising resource use and preventing any single node from becoming a bottleneck.

Another important aspect of the edge layer in the proposed architecture is that it uses a TSN-based communication

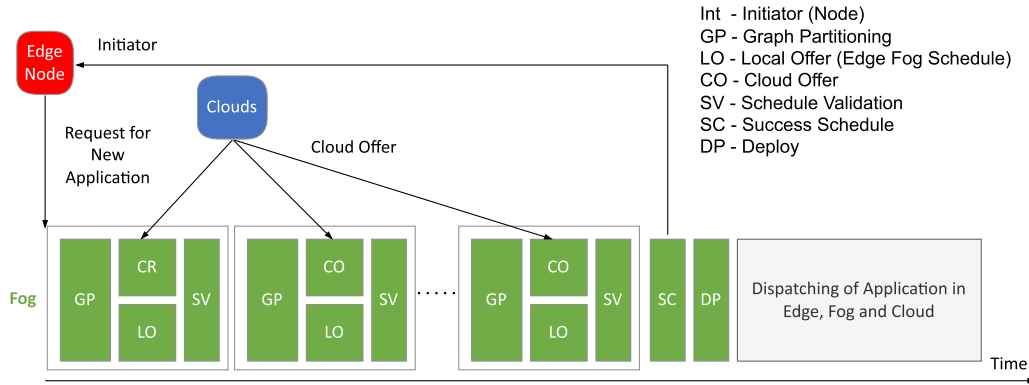


FIGURE 4. Application processing pipeline.

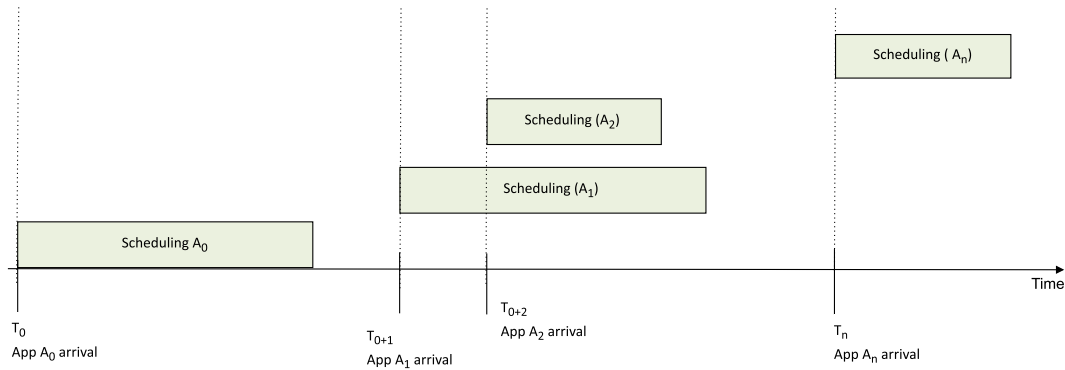


FIGURE 5. Dynamic arrival of applications and scheduling in TTEFC architecture.

approach, making it suitable for real-time and safety-critical tasks.

D. MATHEMATICAL MODEL OF THE THREE-TIER ARCHITECTURE

The proposed TTEFC architecture can be modelled as a graph $G = (V, E)$ where V represents the computing nodes and the routers, and E represents the communication links between these nodes (computing and routers). Each node has specific computational capabilities (only for computational nodes), and the communication between nodes is governed by Time-Sensitive Networking (TSN) and DetNet to ensure deterministic data transfer.

1) PLATFORM MODEL

Let N denote the set of computational nodes, partitioned by layer as

$$N = N_E \cup N_F \cup N_C \quad \text{and} \\ N_E \cap N_F = N_F \cap N_C = N_C \cap N_E = \emptyset,$$

where:

- N_E = set of edge nodes,
- N_F = set of fog nodes,
- N_C = set of cloud nodes.

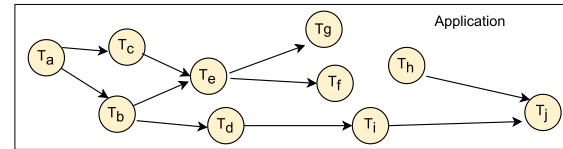


FIGURE 6. Application modelled as DAG.

Each node $r \in N$ has a processing capacity (clock speed) C_r . The (bounded) communication delay between nodes u and v is $D_{u,v}$. We assume a deterministic network fabric (e.g., TSN/DetNet) provides bounded transfer latencies.

The cloud layer is modelled as a opaque abstraction concerning internal scheduling and infrastructure details; it is included via abstracted computational capacity and communication delays to account for its influence on system-level performance metrics such as makespan.

2) APPLICATION MODEL

The application is modelled as a Directed Acyclic Graph (DAG), $T = (J, L)$, where:

- $J = \{J_1, J_2, \dots, J_{|J|}\}$ is the set of tasks in the DAG,
- $L \subseteq J \times J$ is the set of directed precedence edges.

Each edge $(J_i, J_j) \in L$ indicates that task J_j cannot start execution until task J_i has completed. For each task J_j :

- P_i is the base processing time (work) of task J_i ,
- $PN_i \in \{N_E, N_F, N_C\}$ is the preferred execution layer.

3) GRAPH PARTITIONING

The graph partitioning technique is used to divide the task graph into subgraphs, ensuring tasks are mapped to layers while considering dependencies and preferred execution nodes. Let $\mathcal{S} = \{S_1, \dots, S_k\}$ be a family of k subsets of J . The family $\mathcal{S}$ is a partition of J if:

- $\bigcup_{\ell=1}^k S_\ell = J$ and $S_\ell \cap S_m = \emptyset$ for all $\ell \neq m$.

The number k is called the cardinality of the partition (number of subgraphs).

E. SCHEDULING OBJECTIVE AND CONSTRAINTS

The global GA inside the HNGA assigns each subgraph S_ℓ to a layer $\Lambda_\ell \in \{E, F, C\}$ while respecting task preferences when required. The local GA then schedules the tasks of S_ℓ on the processors available in the assigned layer so as to minimise that layer's makespan.

1) INTUITIVE FORMULATION

For a task J_i scheduled on a processor r in its assigned layer, its finish time can be described informally as

$$F_i = \max\left(\max_{p \in \text{Pred}(J_i)} F_p, \text{CTP}[r]\right) + \frac{P_i}{C_r} + \sum_{p \in \text{Pred}(J_i)} D_{p,i}, \quad (3)$$

where $\text{Pred}(J_i)$ are the predecessors of J_i , $\text{CTP}[r]$ is the current-availability time of processor r , and $D_{p,i}$ is the communication delay to deliver the output of J_p to the node executing J_i (zero if on the same node/layer, bounded otherwise).

A clear characterization of the overall makespan M is the maximum finish time over the three layers (edge, fog, cloud):

$$M = \max \left\{ \max_{J_i \in V_E} f_i, \max_{J_i \in V_F} f_i, \max_{J_i \in V_C} f_i \right\}. \quad (4)$$

where V_E, V_F, V_C are the task sets mapped to the edge, fog, and cloud layers, respectively, and f_i is the finish time of task J_i in the precedence-constrained schedule. Inter-layer communication is already accounted for inside each f_i via a bounded delay. The full optimisation model is given in the following section.

2) FORMAL OPTIMIZATION MODEL

We define the makespan as

$$M = \max_{J_i \in J} f_i, \quad (5)$$

where f_i is the finish time of task J_i . This objective is captured via the following optimisation model.

3) DECISION VARIABLES

- $x_{i,r} \in \{0, 1\}$: a binary variable that is 1 if task J_i is assigned to processor $r \in N$, and 0 otherwise.
- $s_i \geq 0$: start time of task J_i .

- $f_i \geq 0$: finish time of task J_i .
- $M \geq 0$: overall makespan of the schedule.
- $y_{ij} \in \{0, 1\}$: ordering variable for tasks J_i and J_j when they are assigned to the same processor.

4) OBJECTIVE FUNCTION

The formal optimisation objective is to minimise the makespan:

$$\min M$$

5) CONSTRAINTS

The optimisation is subject to:

- Assignment constraint (no replication): each task must be assigned to exactly one processor:

$$\sum_{r \in N} x_{i,r} = 1, \quad \forall J_i \in J.$$

- Execution-time constraint: finish time equals start time plus execution time on the chosen processor:

$$f_i \geq s_i + \sum_{r \in N} x_{i,r} \cdot \frac{P_i}{C_r}, \quad \forall J_i \in J.$$

- Precedence and communication constraint: a task J_j can only start after its predecessor J_i has finished and the data has been transferred:

$$s_j \geq f_i + \sum_{r_a \in N} \sum_{r_b \in N} x_{i,r_a} x_{j,r_b} D_{r_a,r_b}, \quad \forall (J_i, J_j) \in L,$$

where $D_{r_a,r_b} = 0$ when communication is intra-node (or same layer, if so modelled), and equals the bounded inter-node (inter-layer) delay otherwise.

- Processor non-overlap constraint: two distinct tasks mapped to the same processor cannot overlap in time.
- Makespan definition: the makespan variable dominates all finish times:

$$M \geq f_i, \quad \forall J_i \in J.$$

Minimising T with these constraints yields a schedule that satisfies precedence and communication requirements and avoids processor conflicts, thus implementing (5) over the three-tier TTEFC architecture. Thus, the HNGA solves a well-defined optimisation problem: it searches over task-to-processor assignments and per-processor orderings to minimise the makespan subject to precedence and communication constraints across the three-tier TTEFC architecture.

F. THREE-TIER SCHEDULING-HIERARCHICAL NESTED GENETIC ALGORITHM

The introduced HNGA is a multi-level optimisation framework that can efficiently schedules tasks across the proposed TTEFC architecture. Traditional task-scheduling approaches struggle with dynamic workloads, resource heterogeneity, and strict real-time constraints. HNGA overcomes these challenges by integrating two levels of genetic optimisation: a GGA for task partitioning and LGA for fine-grained scheduling within each layer.

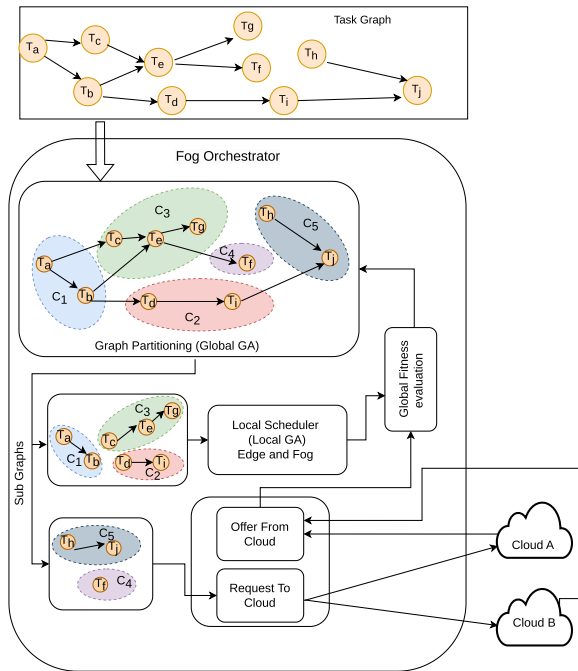


FIGURE 7. Proposed hierarchical nested genetic algorithm.

The GGA is responsible for partitioning the application which is modelled as DAG into smaller subgraphs and then assigns them to the Edge, Fog, or Cloud layers based on computational requirements and data dependencies. Once partitioning is completed, each layer employs an independent LGA to schedule its assigned tasks efficiently. The LGA operates within the assigned layer, optimising task execution thus minimising the makespan. This nested structure allows the system to dynamically adjust to changing workloads, making HNGA well-suited for real-time and safety-critical applications. Figure 7 shows the overall structure of the proposed HNGA, and the pseudo algorithm of HNGA is given in algorithm 1.

By using genetic algorithms at both global and local levels, HNGA provides a scalable and adaptive scheduling mechanism that significantly improves the system performance. The genetic algorithms evolve through selection, crossover, and mutation operations thus ensuring that optimal task distribution, partitions and execution strategies are obtained.

1) GRAPH PARTITIONING FOR TASK DISTRIBUTION-GLOBAL GENETIC ALGORITHM

Effective task distribution is important for minimising execution time and optimising resource utilisation in distributed environments. The proposed HNGA employs graph partitioning techniques to split the application modelled as DAG into smaller subgraphs, ensuring that each task is executed in the most suitable layer of the TTEFC architecture. The GGA is responsible for this partitioning and considers the following factors: execution and data dependencies.

Algorithm 1 Hierarchical Nested Genetic Algorithm (HNGA) for Three-Tier Scheduling

- 1: **Input:** Application DAG $T = (J, L)$, Platform Model $N = N_E \cup N_F \cup N_C$
- 2: **Output:** Optimised task-to-node mapping with minimised makespan
- 3: **Step 1: Global GA - Task Partitioning with Nested Local Scheduling**
- 4: Initialise population of task partitions (subgraphs) $P = \{S_1, S_2, \dots, S_k\}$
- 5: **for** each generation in Global GA **do**
- 6: **for** each partition $S_i \in P$ **do**
- 7: Assign S_i to edge, fog, and cloud layers
- 8: **Invoke Local GA: Task Scheduling in Assigned Layer**
- 9: Initialise population for task scheduling on nodes in the assigned layer
- 10: **for** each generation in Local GA **do**
- 11: Evaluate fitness (makespan in the assigned layer)
- 12: Apply selection, crossover, and mutation operators
- 13: Update population with new schedules
- 14: **end for**
- 15: Select the best schedule for S_i
- 16: **end for**
- 17: Evaluate fitness of each partition using combined makespan + communication delays
- 18: Apply selection, crossover, and mutation operators
- 19: Update population with new partitions
- 20: **end for**
- 21: **Step 2: Return the Optimised Schedule with Minimum Makespan**

The partitioning process begins with DAG analysis, where each task is categorised based on its execution dependencies and data dependencies. The fitness function in GGA evaluates the partitions based on a combination of makespan and inter-layer communication overhead. The algorithm iterates through multiple generations, refining the partitioning until an optimal partition is found. Using graph partitioning, HNGA ensures the workload is evenly distributed, preventing bottlenecks in any single layer while minimising the overall makespan. This approach significantly improves the makespan, allowing seamless coordination between Edge, Fog, and Cloud resources.

Chromosome Representation in GGA The structure of each chromosome consists of four key components:

- Partition – Defines the IDs of the subgraphs.
- Cluster Assigned – Specifies the computational cluster (edge, fog, or cloud) assigned to each partition, considering resource availability and execution constraints.
- Path Index – Indicates the message path between layers, helping optimise inter-layer communication.

- **Message Priority** – Assigns priority levels for message transfers at inter-level to ensure that time-sensitive communications are handled with minimal latency.

This representation enables GGA to explore multiple partitioning strategies, improving the partitions.

The crossover and mutation operations in the GGA play a vital role in refining task partitioning and optimising task distribution across the *Edge, Fog, and Cloud layers*. During crossover, two-parent partitioning schemes exchange a portion of their structure to create offspring with mixed characteristics. This process is applied specifically to the *partition* part of the chromosome, ensuring that task groupings are recombined while preserving dependencies. The mutation operation introduces random modifications to prevent premature convergence and explore diverse partitioning possibilities. The *partition mutation* function in the GGA shuffles partitions. The *layer assignment mutation* function modifies how subgraphs are mapped to different computational layers, considering execution constraints and hardware compatibility. Additionally, *inter-layer communication paths* function in the mutation operation defined in GGA and *message priorities* undergo mutation to explore alternative message routing strategies and optimise latency-sensitive communication. These genetic operators enhance task scheduling efficiency by improving resource allocation, minimising makespan, and reducing inter-layer communication costs. The pseudo algorithm of mutation and crossover in the GGA is shown in algorithm 2.

2) LOCAL GENETIC ALGORITHMS (LGA) FOR SCHEDULING

Once the GGA has partitioned the tasks and assigned them to the appropriate layers, the LGA are activated to determine the optimal scheduling sequence. Each LGA operates independently, optimising task execution order, resource allocation, and deadline adherence within its respective layer. The primary objective of LGA is to minimise makespan while maximising throughput and resource utilisation.

The chromosome of LGA is made up of *Task Order, Processor Allocation, Message Path, and Message Priority*. The *Task Order* specifies tasks in the application. The *Processor Allocation* part maps the processors on which the task can run. It is populated by considering the execution dependencies. The *Message Path* defines the communication pathways at intra-level. and the *Message Priority* indicates the priority levels of message transfers. This allows the system to ensure timely communication between tasks, minimising delays caused by congestion and resource contention. The LGA operates by evaluating the fitness of each chromosome, which is based on optimising makespan and minimising communication delays. The algorithm employs standard genetic operators, such as selection, crossover, and mutation, to explore the solution space and converge towards an optimal or near-optimal scheduling solution for the tasks within the assigned layer. The pseudo mutation and crossover operator logic is provided in the algorithm 3

Algorithm 2 Crossover and Mutation - Global Genetic Algorithm

- 1: **Input:** Population of chromosomes, each representing a partitioning scheme
 - 2: **Output:** Optimised partitioning scheme and final schedule
 - 3: **Step 1: Crossover Operation**
 - 4: Select pairs of parent chromosomes from the population.
 - 5: Identify the *partitioning segment* within each chromosome.
 - 6: Perform swapping a section of the partitioning structure between parents.
 - 7: Generate offspring with mixed task groupings.
 - 8: **Step 2: Mutation Operation**
 - 9: **Partition Mutation:**
 - 10: For each individual, extract its partitioning segment.
 - 11: Randomly shuffle partitions to get a new structure.
 - 12: **Layer Assignment Mutation:**
 - 13: Extract the section representing *partition*.
 - 14: For each subgraph, check hardware constraints and data dependencies and assign it to a suitable computational layer.
 - 15: **Inter-Layer Communication Mutation:**
 - 16: Modify message paths between layers by shuffling path index.
 - 17: **Message Priority Mutation:**
 - 18: Adjust the priority levels of inter-layer messages to optimise communication timing.
 - 19: **Step 3: Repeat Crossover and Mutation**
 - 20: Repeat the crossover and mutation operations until convergence criteria are met or a near-optimal partitioning scheme is found.
 - 21: **Return:** Optimised partitioning scheme for task scheduling
-

By executing LGAs independently in each tier, the scheduling process remains modular and scalable, enabling efficient adaptation to varying workloads. The nested structure of HNGA ensures that task execution is not only optimised at a global level (GGA) but also refined at the local level (LGA), leading to faster and more reliable scheduling decisions.

3) APPLICATION OF TTEFC AND HNGA

The proposed TTEFC architecture and HNGA can be applied in real-time autonomous driving systems, where vehicles need to process sensor data, make quick decisions, and communicate with infrastructure and other vehicles while ensuring safety and efficiency. At the edge layer, each vehicle processes real-time sensor data from LiDAR, cameras, and radar to detect obstacles, recognise pedestrians, and make immediate driving decisions like braking or lane changes. Since these tasks require fast responses, they are executed on onboard hardware (edge layer). The fog layer, consisting

Algorithm 3 Crossover and Mutation - Local Genetic Algorithm

- 1: **Input:** Population of chromosomes, each representing a task scheduling scheme
- 2: **Output:** Optimal scheduling scheme with improved local makespan
- 3: **Step 1: Crossover Operation**
- 4: Select pairs of parent chromosomes from the population.
- 5: Identify the *task order segment* within each chromosome.
- 6: Perform *one-point crossover* on the task order segment.
- 7: Generate offspring with a mixed execution sequence.
- 8: **Step 2: Mutation Operations**
- 9: **Task Order Mutation:**
- 10: Select a chromosome and extract its *task order segment*.
- 11: Randomly swap two tasks within the sequence.
- 12: **Processor Allocation Mutation:**
- 13: Extract the *processor allocation segment*.
- 14: For each task, check hardware constraints and assign a suitable processor.
- 15: **Path Index Mutation:**
- 16: Extract the *message path segment*.
- 17: With probability p_m , replace the current path with a randomly selected alternative.
- 18: **Message Priority Mutation:**
- 19: Extract the *message priority segment*.
- 20: Shuffle the priority values while ensuring unique identifiers.
- 21: **Step 3: Repeat Crossover and Mutation**
- 22: Repeat the crossover and mutation operations until convergence criteria are met or an optimal scheduling scheme is found.
- 23: **Return:** Optimal task scheduling scheme with minimised makespan.

of roadside units (RSUs), manages vehicle communication and coordinates traffic by handling tasks like lane merging and traffic signal control. The cloud layer is responsible for processing large-scale data, such as analysing traffic patterns, optimising long-term routes, and improving AI models

Using TTEFC and HNGA, autonomous vehicles can operate more reliably and respond faster to changing conditions. This approach helps improve real-time decision-making, resource management, and safety in intelligent transportation systems.

4) COMPUTATIONAL COMPLEXITY

The computational complexity of the Hierarchical Nested Genetic Algorithm (HNGA) is driven by its nested evaluation structure. In HNGA, the Global GA (GGA) evaluates its individuals; each evaluation triggers one Local GA (LGA) run per subgraph. During a single LGA fitness evaluation, the makespan of the assigned subgraph is computed via a forward pass over a topological order with per-processor

TABLE 2. HNGA hyperparameters for GGA and LGA.

Parameter	Value
Population size	20
Generations	25
Crossover probability	0.8
Mutation probability	0.4
Tournament size	3

TABLE 3. PSO baseline hyperparameters.

Parameter	Value
Seed	42
Swarm size (N_p)	20
Iterations (T)	25
Inertia (w)	0.729
Cognitive (c_1)	1.45
Social (c_2)	1.45
Velocity clamp ($v_{\max}$)	3

availability updates, which takes $\Theta(n_\ell + e_\ell)$ time for subgraph S_ℓ . Consequently, the cost of each LGA run scales with its generations G_L and population size P_L . The total cost of one GGA evaluation is the sum of these costs across all subgraphs defined by that chromosome. Let $n = |J|$ be the number of tasks and $e = |L|$ the number of precedence edges.

Let P_G, G_G be the GGA population size and generations; P_L, G_L the LGA population size and generations. One GGA evaluation therefore costs $\Theta(G_L P_L (n + e))$, so over G_G generations and P_G individuals:

$$T_{\text{HNGA}} = \Theta(G_G P_G G_L P_L (n + e)).$$

IV. EXPERIMENTS

In this study, applications (DAGs) were created using the Stanford Network Analysis Platform (SNAP) [33]. For a robust evaluation, we utilised the computational resources of the University of Siegen's OMNI cluster [34] to generate the application models and run the proposed algorithms. The implementation of HNGA was carried out using the Distributed Evolutionary Algorithms in Python (DEAP) library [35].

To successfully run the HNGA in the proposed architecture, we needed to model the Cloud layer, which was treated as a opaque abstraction in terms of internal scheduling strategies and resource availability, reflecting the abstracted and often opaque nature of commercial Cloud services. Unlike the Edge and Fog layers, where we have more control over the hardware and scheduling mechanisms, the Cloud environment offers limited visibility and control, as real Cloud infrastructures are managed by third-party providers with proprietary configurations. Hence to get the cloud offers and successfully evaluate the HNGA we run the algorithms in the modelled cloud layer along with the fog and edge layers. Additionally we also assume that a deterministic networking fabric (TSN/DetNet) is available, providing deterministic communication between different layers. Thus we do not synthesise TSN/DetNet schedules in this work.

To evaluate the performance of the proposed HNGA task scheduling algorithm, we conducted a comparative study

TABLE 4. Application scenarios and their task details.

Scenarios	Number of Tasks in Application DAG
Scenario 1	100
Scenario 2	200
Scenario 3	250
Scenario 4	500
Scenario 5	750

against several well-known scheduling algorithms, including Min-Min Particle Swarm Optimisation (PSO) and Heterogeneous Earliest Finish Time (HEFT). Our experimental setup across the different algorithms was kept identical to ensure a fair comparison, which means that the same application models (DAGs) and platform models were used. This was done to confirm that any observed performance differences were only due to the different scheduling approaches. The key metrics used to evaluate the algorithms is:

- **Total Execution Time (Makespan):** This evaluation metric measures the overall time required to execute all tasks in the DAG in the given platform model. A smaller makespan indicates better scheduling efficiency.

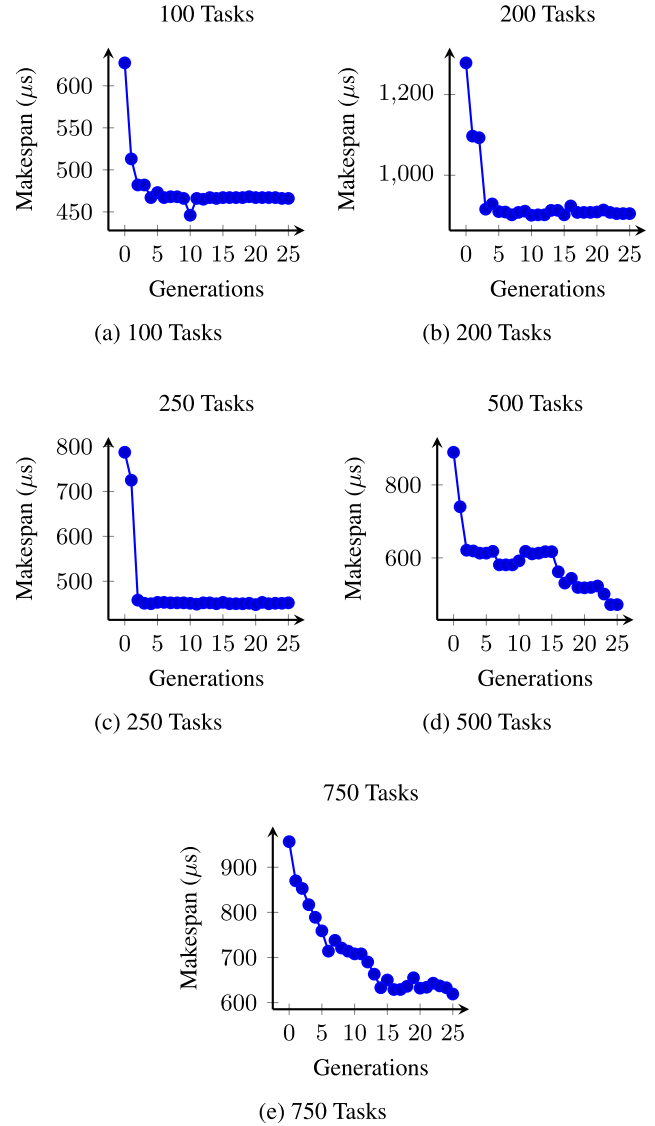
Table 4 summarises the different application scenarios used in this research. Each application scenario corresponds to a DAG. The “Application” column in the table indicates the number of tasks within each DAG.

The tables 2 and 3 report the full hyperparameters for both HNGA and PSO baselines, respectively. In PSO, the cognitive coefficient c_1 pulls a particle toward its own best-known position, while the social coefficient c_2 pulls it toward the swarm’s best.

By comparing the proposed approach HNGA with other scheduling methods under these controlled conditions, we demonstrated its effectiveness in reducing execution time while achieving better load balancing across the TTEFC architecture.

A. VALIDATION ON REAL-TIME DATASET

Publicly available real-time datasets for autonomous and other safety-critical applications are largely unavailable due to intellectual property restrictions. To address this gap, we adopted the methodology mentioned in the paper [36] and instantiated our synthesised DAGs with realistic timing attributes. Each job in the DAG was assigned the following Periods, deadlines, execution time and message size. Periods were given the classes 10, 20, 50, 100 ms, deadlines followed the implicit model $D = \beta P$ (with $\beta = 1.0$), and execution times were derived from the published averages and worst-case bounds in the paper [36]. Message sizes were chosen from the following set: 1, 2, 4, 8 bytes, as defined in the paper [36]. This mapping helped us preserve our original SNAP DAG structure and processor constraints. Thus, providing an automotive-like workload in a reproducible, IP-free, real-time dataset suitable for deadline-driven evaluation.

**FIGURE 8. Makespan vs. number of generations for different task sizes, showing the convergence of the optimization process.**

V. RESULTS

In this study, we evaluated the proposed HNGA performance for task scheduling within TTEFC architecture. We compared HNGA with established algorithms such as Min-Min, Particle Swarm Optimisation (PSO) and Heterogeneous Earliest Finish Time (HEFT). The results demonstrate that HNGA outperforms these algorithms in terms of both makespan and fitness values across various task sizes.

The proposed HNGA framework combines graph partitioning (GP) with genetic algorithms (GAs), which is crucial to achieving these results. The proposed HNGA improved the objective function (minimise the makespan) significantly as generations progressed. For a task size of 100, the fitness value decreased to 466.0 (μs) over generation, highlighting the algorithm’s effectiveness in minimising scheduling costs. Similarly, for larger task sizes the fitness values also showed a decreasing trend showing the algorithm’s robustness across various scenarios. Figure 8 shows the convergence

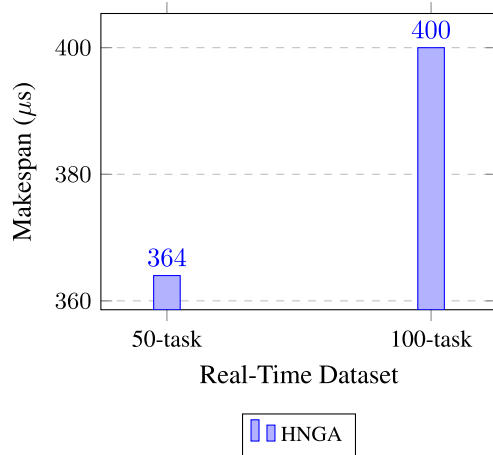


FIGURE 9. Best global makespan achieved by HNGA on real-time DAGs.

of makespan (fitness value) for different scenarios for the proposed algorithm HNGA.

Figure 11 shows the comparison of HNGA versus Min-Min, PSO, and HEFT. Tables 6 and 7 report the makespans and the percentage reductions of HNGA relative to each baseline, respectively. For 100 tasks, HNGA achieves 466 (μs) versus 722 (μs) for Min-Min, 1872 (μs) for PSO, and 685 (μs) for HEFT, corresponding to reductions of 35.5%, 75.1%, and 31.9%, respectively. Similarly for larger DAGs: for 200 tasks, 905 (μs) versus 1935, 7159, and 2006 (μs) (reductions of 53.3%, 87.4%, and 54.9%); for 250 tasks, 452 (μs) versus 688, 1650, and 606 (μs) (34.3%, 72.6%, and 25.4%); for 500 tasks, 472 (μs) versus 857, 2574, and 752 (μs) (44.9%, 81.7%, and 37.3%); and for 750 tasks, 619 (μs) versus 706, 3335, and 654 (μs) (12.3%, 81.5%, and 5.4%). Across all five scenarios, HNGA consistently outperforms Min-Min and PSO, and it improves over HEFT in every case (5.4%–54.9% reduction), underscoring both scalability and effectiveness in heterogeneous Edge-Fog-Cloud settings.

A. RESULT ON REAL-TIME DATASET

Figure 9 shows the best global makespan achieved by HNGA for the 50-task and 100-task cases (364 and 400 respectively). The plot of makespan versus task count shows that HNGA remains effective when applied to realistic automotive timing constraints. This real-time validation complements the broader statistical analysis conducted on SNAP-generated DAGs, demonstrating both the scalability and applicability of HNGA in safety-critical scenarios.

B. STATISTICAL ANALYSIS ON MULTIPLE RUNS

To assess robustness across seeds, we executed HNGA on the $|V| = 100$ application for 25 independent runs and report descriptive statistics. Refer table 5. The best makespan per run shows median = 476.00 μs with interquartile range (IQR) = 21.00 μs , mean = 482.56 μs , std = 27.74 μs , CoV = 5.75%, and 95% confidence interval (CI) for the mean = [471.11, 494.01] μs . The across-run best (min) was

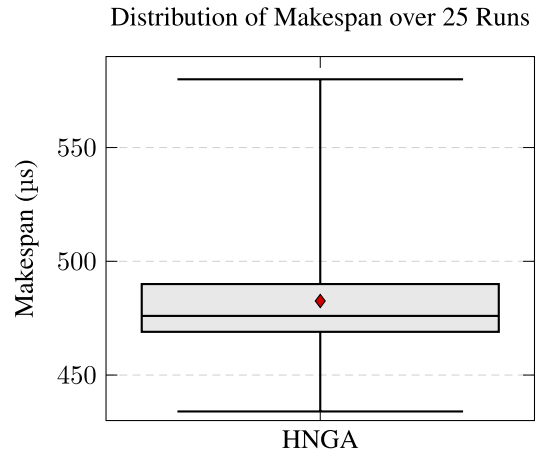


FIGURE 10. Makespan over 25 HNGA runs. Box shows median and IQR; whiskers show range; diamond marks the mean.

TABLE 5. Statistics for HNGA on $|V| = 100$ application (25 runs).

Metric	Value
Median	476.00 μs
Mean	482.56 μs
std	27.74 μs
Best makespan (min / max)	434.00/580.00 μs
Coefficient of Variation (CoV)	5.75%
95% Confidence Interval (CI) (mean makespan)	[471.11, 494.01] μs
Fitness improvement (median)	9.33%
Fitness improvement (mean $\pm$ std)	9.77 $\pm$ 5.68%

TABLE 6. Makespan (in μs) of HNGA and baseline algorithms.

Scenario	HNGA	Min-Min	PSO	HEFT
1 (100 Tasks)	466	722	1872	685
2 (200 Tasks)	905	1935	7159	2006
3 (250 Tasks)	452	688	1650	606
4 (500 Tasks)	472	857	2574	752
5 (750 Tasks)	619	706	3335	654

TABLE 7. Reduction in makespan (%) achieved by HNGA compared to baselines.

Scenario	vs Min-Min	vs PSO	vs HEFT
1 (100 Tasks)	35.5	75.1	31.9
2 (200 Tasks)	53.3	87.4	54.9
3 (250 Tasks)	34.3	72.6	25.4
4 (500 Tasks)	44.9	81.7	37.3
5 (750 Tasks)	12.3	81.5	5.4

434.00 μs and the worst (max) 580.00 μs , indicating low dispersion across seeds. Convergence analysis shows the best solution first appears at generation 16 (median, IQR = 11; min = 2, max = 25), i.e. roughly half the runs reach the final best by generation ≤ 15 (48%), and 16% do so by ≤ 10 generations. Finally, the evolutionary process improved population quality, with the final mean fitness increasing by a median of 9.33% (overall mean $9.77 \pm 5.68\%$). The box plot in Figure 10 confirms the tight dispersion of makespan across 25 runs. Overall, the low CoV for makespan, tight IQR, and stable convergence indicate that HNGA's performance is consistent across runs and computationally reliable for repeated evaluations.

These findings highlight the effectiveness of the proposed HNGA. The integration of graph partitioning with concurrent

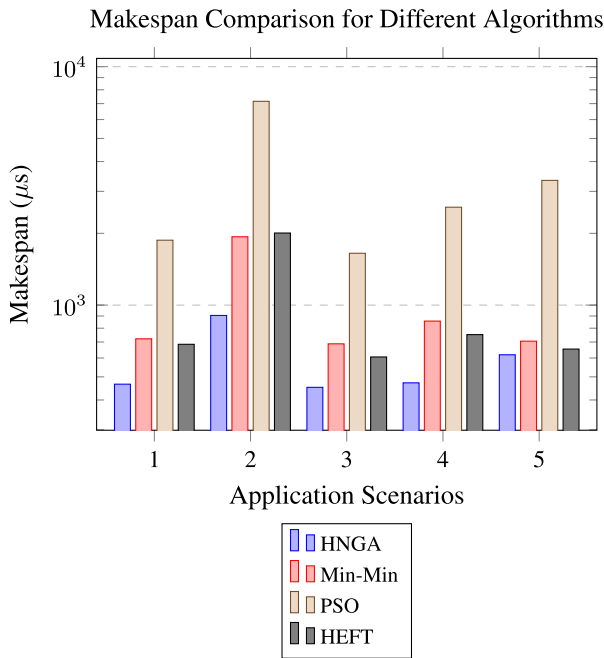


FIGURE 11. Makespan Comparison for HNGA, Min-Min, PSO and HEFT.

scheduling across the TTEFC architecture enhanced both makespan reduction and fitness value improvement. This makes the proposed HNGA a well-suited solution for optimal task scheduling in complex, multi-layered TTEFC architectures, particularly in real-time, safety-critical applications.

VI. CONCLUSION

In this paper, we introduced the HNGA for task scheduling in TTEFC architecture. Our approach integrates graph partitioning with genetic algorithms to optimise task allocation while ensuring strict timing constraints are met. Experimental results demonstrate that HNGA outperforms traditional scheduling algorithms, such as Min-Min, Particle Swarm Optimisation (PSO) and Heterogeneous Earliest Finish Time (HEFT), particularly in terms of reducing makespan. Across five scenarios, HNGA reduced makespan by 12.3–53.3% versus Min-Min, 72.6–87.4% versus PSO and 5.4%–54.9% reduction versus HEFT, with consistent convergence trends.

The primary advantage of the proposed HNGA lies in its hierarchical structure. The GGA efficiently partitions the application model into smaller subgraphs, which are then scheduled by LGA within the respective architecture layers. This two-level approach ensures tasks are effectively distributed across the edge, fog, and cloud resources, promoting better load balancing and minimising makespan. The combination of graph partitioning and local task scheduling allows for optimal task allocation, even when handling varying task sizes and complexities.

The TTEFC architecture plays a key role in enabling scalable and efficient resource management. By utilising edge, fog, and cloud layers, TTEFC ensures that tasks are processed in the most suitable layer, optimising both response time and computational efficiency. The ability

to dynamically allocate tasks across these layers improves overall system performance, making it highly suitable for real-time applications that require fast decision-making, such as autonomous driving and smart city infrastructure.

Future work will focus on extending HNGA to handle dynamic environments, including real-time task arrival, task re-allocation in case of resource failure, and the integration of energy efficiency into the scheduling process. Additionally, fault tolerance using meta-scheduling will be incorporated to enhance system resilience by leveraging precomputed schedules for rapid adaptation to failures. Furthermore, reliability assessment techniques will be developed to evaluate the robustness of the scheduling framework under varying operational conditions. Additionally TSN/DetNet scheduling with GCL generation to provide deterministic communication also will be addressed in future works. These improvements will enhance the applicability of HNGA in complex, real-world scenarios, making it a more effective solution for time-sensitive, resource-constrained applications.

ACKNOWLEDGMENT

This work was supported by the EdgeAI-Trust project, which is funded within the Chips Joint Undertaking (Chips JU), a public-private partnership in collaboration with the European Union's Horizon Europe research and innovation program and National Authorities under grant agreement number 101139892.

With funding from the:



Federal Ministry
of Research, Technology
and Space

REFERENCES

- [1] K. Sathupadi, S. Achar, S. V. Bhaskaran, N. Faruqui, M. Abdullah-Al-Wadud, and J. Uddin, "Edge-cloud synergy for AI-enhanced sensor network data: A real-time predictive maintenance framework," *Sensors*, vol. 24, no. 24, p. 7918, Dec. 2024. [Online]. Available: <https://www.mdpi.com/1424-8220/24/24/7918>
- [2] J. H. Lala and R. E. Harper, "Architectural principles for safety-critical real-time applications," *Proc. IEEE*, vol. 82, no. 1, pp. 25–40, 1994.
- [3] B. Yaylymov, F. Rezabek, and K. Holzinger, "Deterministic networking-DetNet," *Network*, vol. 65, pp. 65–70, Nov. 2022, doi: [10.2313/NET-2022-11-1_12](https://doi.org/10.2313/NET-2022-11-1_12).
- [4] N. Finn, P. Thubert, B. Varga, and J. Farkas, "RFC 8655: Deterministic networking architecture," RFC Editor, RFC 8655, Tech. Rep., Oct. 2019. [Online]. Available: <https://www.rfc-editor.org/info/rfc8655>
- [5] P. Kochovski, R. Sakellariou, M. Bajec, P. Drobintsev, and V. Stankovski, "An architecture and stochastic method for database container placement in the edge-fog-cloud continuum," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, May 2019, pp. 396–405.
- [6] L. Tong, Y. Li, and W. Gao, "A hierarchical edge cloud architecture for mobile computing," in *Proc. 35th Annu. IEEE Int. Conf. Comput. Commun. (IEEE INFOCOM)*, Apr. 2016, pp. 1–9.
- [7] N. Mohan and J. Kangasharju, "Edge-fog cloud: A distributed cloud for Internet of Things computations," in *Proc. Cloudification Internet Things (CIoT)*, Nov. 2016, pp. 1–6.
- [8] L. Hou, S. Zhao, X. Xiong, K. Zheng, P. Chatzimisios, M. S. Hossain, and W. Xiang, "Internet of Things cloud: Architecture and implementation," *IEEE Commun. Mag.*, vol. 54, no. 12, pp. 32–39, Dec. 2016.

- [9] J. Dizdarević, F. Carpio, A. Jukan, and X. Masip-Bruin, "A survey of communication protocols for Internet of Things and related challenges of fog and cloud computing integration," *ACM Comput. Surveys*, vol. 51, no. 6, pp. 1–29, Nov. 2019.
- [10] S. Khrijji, Y. Benbelgacem, R. Chéour, D. E. Houssaini, and O. Kanoun, "Design and implementation of a cloud-based event-driven architecture for real-time data processing in wireless sensor networks," *J. Supercomput.*, vol. 78, no. 3, pp. 3374–3401, Feb. 2022.
- [11] S. Malik and F. Huet, "Adaptive fault tolerance in real time cloud computing," in *Proc. IEEE World Congr. Services*, Mar. 2011, pp. 280–287.
- [12] A. Koubaa and B. Qureshi, "Dronetrack: Cloud-based real-time object tracking using unmanned aerial vehicles over the internet," *IEEE Access*, vol. 6, pp. 13810–13824, 2018.
- [13] V. Kochar and A. Sarkar, "Real time resource allocation on a dynamic two level symbiotic fog architecture," in *Proc. 6th Int. Symp. Embedded Comput. Syst. Design (ISED)*, Dec. 2016, pp. 49–55.
- [14] S. Liu, G. Quan, and S. Ren, "On-line scheduling of real-time services for cloud computing," in *Proc. 6th World Congr. Services*, Feb. 2010, pp. 459–464.
- [15] H. Kopetz and S. Poledna, "In-vehicle real-time fog computing," in *Proc. 46th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. Workshop (DSN-W)*, Jul. 2016, pp. 162–167.
- [16] M. L. Raagaard, P. Pop, M. Gutiérrez, and W. Steiner, "Runtime reconfiguration of time-sensitive networking (TSN) schedules for fog computing," in *Proc. IEEE Fog World Congr. (FWC)*, Oct. 2017, pp. 1–6.
- [17] M. R. Alizadeh, V. Khajehvand, A. M. Rahmani, and E. Akbari, "Task scheduling approaches in fog computing: A systematic review," *Int. J. Commun. Syst.*, vol. 33, no. 16, p. e4583, Nov. 2020.
- [18] F. R. Shahidani, A. Ghasemi, A. T. Haghighat, and A. Keshavarzi, "Task scheduling in edge-fog-cloud architecture: A multi-objective load balancing approach using reinforcement learning algorithm," *Computing*, vol. 105, no. 6, pp. 1337–1359, Jun. 2023, doi: [10.1007/s00607-022-01147-5](https://doi.org/10.1007/s00607-022-01147-5).
- [19] C. Yin, Q. Fang, H. Li, Y. Peng, X. Xu, and D. Tang, "An optimized resource scheduling algorithm based on GA and ACO algorithm in fog computing," *J. Supercomput.*, vol. 80, no. 3, pp. 4248–4285, Feb. 2024.
- [20] M. N. Abdulredha, B. A. Attea, and A. J. Jabir, "An evolutionary algorithm for task scheduling problem in the cloud-fog environment," *J. Phys., Conf. Ser.*, vol. 1963, no. 1, Jul. 2021, Art. no. 012044.
- [21] F. Hoseiny, S. Azizi, M. Shojafar, F. Ahmadiazar, and R. Tafazolli, "PGA: A priority-aware genetic algorithm for task scheduling in heterogeneous fog-cloud computing," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, May 2021, pp. 1–6.
- [22] I. M. Ali, K. M. Sallam, N. Moustafa, R. Chakraborty, M. Ryan, and K. R. Choo, "An automated task scheduling model using non-dominated sorting genetic algorithm II for fog-cloud systems," *IEEE Trans. Cloud Comput.*, vol. 10, no. 4, pp. 2294–2308, Oct. 2022.
- [23] X.-Q. Pham and E.-N. Huh, "Towards task scheduling in a cloud-fog computing system," in *Proc. 18th Asia Pacific Netw. Oper. Manage. Symp. (APNOMS)*, Oct. 2016, pp. 1–4.
- [24] T. Choudhari, M. Moh, and T.-S. Moh, "Prioritized task scheduling in fog computing," in *Proc. ACMSE Conf.*, Mar. 2018, pp. 1–8, doi: [10.1145/3190645.3190699](https://doi.org/10.1145/3190645.3190699).
- [25] J. C. Guevara and N. L. S. da Fonseca, "Task scheduling in cloud-fog computing systems," *Peer-Peer Netw. Appl.*, vol. 14, no. 2, pp. 962–977, Mar. 2021.
- [26] A. S. Abohamama, A. El-Ghamry, and E. Hamouda, "Real-time task scheduling algorithm for IoT-based applications in the cloud-fog environment," *J. Netw. Syst. Manage.*, vol. 30, no. 4, pp. 1–35, Oct. 2022. [Online]. Available: <https://link.springer.com/article/10.1007/s10922-022-09664-6>
- [27] J. Paulachan, D. Onwuchekwa, and R. Obermaisser, "Task scheduling in multi-cloud environments: A graph partitioning approach enhanced by nested genetic algorithms," in *Proc. 11th Int. Conf. Internet Things, Syst., Manage. Secur. (IOTSMS)*, Sep. 2024, pp. 177–184.
- [28] O. Hekal, D. Onwuchekwa, and R. Obermaisser, "Incremental scheduling using genetic algorithm," in *Proc. Int. Symp. ELMAR*, Sep. 2024, pp. 269–275.
- [29] J. Paulachan, D. Onwuchekwa, and R. Obermaisser, "Incremental concurrent task scheduling in multi-cluster systems: Utilising graph partitioning and dynamic genome length genetic algorithm," *Proc. Comput. Sci.*, vol. 257, pp. 927–936, Aug. 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050925008567>
- [30] V. Kumar, L. Singh, and A. K. Tripathi, "Reliability analysis of safety-critical and control systems: A state-of-the-art review," *IET Softw.*, vol. 12, no. 1, pp. 1–18, Feb. 2018.
- [31] I. Koren and C. Krishna, *Fault-Tolerant Systems*. San Mateo, CA, USA: Morgan Kaufmann, 2020.
- [32] S. Majidi, R. Obermaisser, S. Wasala, and M. Qosja, "Fault-tolerant scheduler with genetic algorithm for safety-critical time-triggered systems of systems," in *Proc. IEEE 18th Int. Conf. Ind. Informat. (INDIN)*, vol. 1, Jul. 2020, pp. 259–264.
- [33] J. Leskovec and R. Sosič, "SNAP: A general-purpose network analysis and graph-mining library," *ACM Trans. Intell. Syst. Technol.*, vol. 8, no. 1, pp. 1–20, Jan. 2017.
- [34] Universität Siegen. Accessed: Jan. 2024. [Online]. Available: <https://cluster.uni-siegen.de>
- [35] J. Kim and S. Yoo, "Software review: DEAP (Distributed evolutionary algorithm in Python) library," *Genetic Program. Evolvable Mach.*, vol. 20, no. 1, pp. 139–142, Mar. 2019.
- [36] S. Kramer, D. Ziegenbein, and A. Hamann, "Real world automotive benchmarks for free," in *Proc. 6th Int. Workshop Anal. Tools Methodologies Embedded Real-Time Syst. (WATERS)*, 2015, pp. 4–3.



JOSEPAUL PAULACHAN received the M.Sc. degree in mechatronics from the University of Siegen, Germany, where he is currently pursuing the Ph.D. degree with the Chair for Embedded Systems. His research focuses on task scheduling, graph partitioning, and reliability in heterogeneous edge-fog-cloud systems, with emphasis on safety-critical and real-time applications. His expertise includes FPGA design and verification, real-time operating systems, and deterministic networking.



OMAR HEKAL received the M.Sc. degree in mechatronics from the University of Siegen, Germany, where he is currently pursuing the Ph.D. degree with the Chair for Embedded Systems. He is a Research Assistant with the Chair for Embedded Systems, University of Siegen. His research focuses on scheduling for time-triggered systems and adaptation techniques for safety-critical systems.



DANIEL ONWUCHEKWA received the Ph.D. degree in electrical engineering and computer science from the University of Siegen, Germany, in 2020. He is currently a Postdoctoral Researcher and a Project Manager with the Division of Embedded Systems, University of Siegen. His doctoral research, supervised by Prof. Roman Obermaisser, focused on developing a fault injection framework for time-triggered systems. He has contributed to several European research projects, including SAFE4RAIL, Ecomobility, and FRACTAL, where he was a Work Package Leader. His current research interests include system architectures for distributed embedded real-time systems, fault-tolerant design, and embedded artificial intelligence (AI) in real-time embedded systems.



ROMAN OBERMAISSER received the master's degree in computer science from Vienna University of Technology, in 2001, the Ph.D. degree, in 2004, under Prof. Hermann Kopetz, and the Habilitation (Venia docendi) degree in technical computer science, in 2009. He is currently a Full Professor with the Division for Embedded Systems, University of Siegen, Germany. His research focuses on system architectures for distributed embedded real-time systems. He is the author of numerous journal and conference publications and a book on an integrated time-triggered architecture (Springer). He has participated in many EU projects (e.g., SAFEPOWER, universAAL, DECOS, and NextTTA) and coordinated the projects DREAMS, GENESYS, and ACROSS.

...